# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a7dd5272404b24b16.jsonl`
- SHA-256: `c991059d59ce814c07950a6c9216d2324a79bfcf953b2ca3d1c9fd1099d7894d`
- Source modified: `2026-07-06T19:19:42+00:00`
- Imported at: `2026-07-08T16:00:12+00:00`
- project: `wf_e1b943dd-dac`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

## Transcript

### 1. user (2026-07-06T19:18:37.648Z)

```
Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR
(branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F
design doc for a
four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content
safety / Tier 4
deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up
issues F1-F7.
No code changes. Read the ADR:
  git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-
pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design
contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the
codebase and the
other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C
C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem,
Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate
should_ingest_source,
delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler:
watched gate -> dedup
-> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track
C (delivery_attempts)
just merged.

Ground rules:
- READ actual files, cite ADR section / file:line. You MAY run
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does
FilterConfig
  ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text
populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies,
not merge-blockers.
  Severity: high (a claim that is false, or a design gap that would break an F-phase / cause
data loss), medium
  (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example,
minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for
implementation.
```

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if
unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (cross_adr): Tier 3 "Source allowlist" is dead code ? it duplicates sources.watched (ADR 0009 / Track E) and can never fire
severity high | section Security stages ? Tier 3, item 1 ("Source allowlist"); phasing F3
summary: Tier 3's first stage "reject items from sources not in the allowlist (trusted provider_source_ids)" reimplements the watched opt-in gate. The pipeline runs "after dedup, before delivery," but the handler's watched gate runs BEFORE dedup and returns early for any unwatched/unknown source. So an item from a non-allowlisted source never reaches the pipeline ? the allowlist stage sees only already-watched sources and rejects nothing new.
why_it_matters: F3 as specified ships a no-op stage. Two overlapping opt-in source-gates (watched vs allowlist) is exactly the conflation the umbrella exists to dissolve, and it will confuse users: an unwatched source is invisible, so an allowlist can only ever be a SUBSET of watched sources with no additional effect. The design needs to either drop the stage or redefine it as something watched doesn't already do (e.g. per-route/per-policy source restriction distinct from account-level ingestion).
evidence: telethon_client.py:189 `if not await db.should_ingest_source(conn, user.id, channel_id): return  # skip unwatched/unknown source` ? this early return precedes all dedup/evaluate/deliver. ADR 0009 Decision §1 defines watched as opt-in DEFAULT 0, gated "at the top... before any dedup or filter work." Track E merged (sources.watched, should_ingest_source). ADR 0012 places the pipeline "after dedup and before delivery," strictly downstream of the watched gate.
rec: State the overlap explicitly and resolve it: either remove the Tier-3 source-allowlist stage (watched already is the source gate), or scope it to something orthogonal ? e.g. a per-policy/per-route source restriction for multi-route accounts (ADR 0009 routing_rules), which is genuinely not covered by the account-wide watched flag. Update F3's scope accordingly.

**2. assistant (2026-07-06T19:18:41.031Z)**

I'll verify this finding by reading the ADR section, the handler code, and ADR 0009.

**3. user (2026-07-06T19:18:43.305Z)**

# ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

## Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

### What we have today (post-Track-E)

| Concept | Current state |
|---|---|

| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig` blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan. |

### What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

## Decision

### Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
????????????????????????????????????????????????????????????????
?                    Policy Pipeline                           ?
?                                                              ?
?  Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?     duration, resolution, size, codec, mime, GIF/round       ?
?     ? extends today's FilterEngine                           ?
?                                                               ?
?  Tier 2 ? Topic preferences  (opt-in, metadata-only)         ?
?     source allowlist, keyword match, channel topic tags       ?
?     ? user-expressed interests filter the stream              ?
?                                                               ?
?  Tier 3 ? Content safety  (opt-in, metadata + light I/O)     ?
?     URL/domain reputation, file metadata sanity,              ?
?     caption text scanning                                     ?
?     ? cheap checks; no download                               ?
?                                                               ?
?  Tier 4 ? Deep inspection  (opt-in, heavy I/O)               ?
?     ffprobe, perceptual hash, downloaded content scan         ?
?     ? gated behind explicit user opt-in + budget              ?
????????????????????????????????????????????????????????????????
```

### Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem        # from ADR 0011
    source: Source                   # the source it came from
    policy: Policy                   # the owning policy config

@dataclass
class StageResult:
    passed: bool
    reason: str                      # human-readable explanation
```

```
    stage_name: str                  # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict)  # for explainability UI

class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

### Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any"  // "any" = match any include topic; "all" = match all
  }
}
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

### Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider_source_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

### Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

### Policy evaluation flow (post-dedup, pre-delivery)

```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results  # short-circuit
    return True, results
```

The caller (telethon_client or future Bot API handler) records the
results via `delivery_attempts.last_error` (for failed items) or a new
`policy_decisions` table (for explainability ? deferred to implementation).

#### Schema ? minimal changes

**No schema migration** is required for this design. The `policies` table
already holds a `config_json` JSON blob. The pipeline is a code concept:

- `policies.config_json` gains optional `topics`, `allowlist`,
  `security_checks`, `deep_inspect` keys ? all backwards-compatible with
  the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy_decisions` table (deferred) would store the explanation
  trace for debugging: `(content_item_id, policy_id, stage_name, passed,
  reason, metadata_json, evaluated_at)`.

#### Implementation phasing (future issues)

| Phase | Scope | Dependencies |
|---|---|---|
| **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler | None |
| **F2** ? Topic preferences | `topics` config parsing, source title/caption keyword matching | F1 |
| **F3** ? Source allowlist | `allowlist` config, skip non-allowlisted sources | F1 |
| **F4** ? URL/domain reputation | Domain blocklist, caption link extraction | F1 |
| **F5** ? Deep inspection | ffprobe/perceptual hash, download budget enforcement | F1, ADR 0003 waiver |
| **F6** ? Policy decisions table | Audit trail storage, explainability UI | F1 |
| **F7** ? Explainability endpoint | `/why <content_item_id>` command showing the explanation trace | F6 |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR
only defines the **design contract** the phases code against.

## Consequences

**Positive**
- Unifies topic, security, and quality gates into a single explainable
  framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for
  items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without
  additional DB lookups at decision time.
- Backwards-compatible: `policies.config_json` already exists; adding
  optional keys requires no migration.
- Each implementation phase is independently shippable behind feature
  flags.

**Negative**

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same
  interface, which may be awkward for deeply different checks
  (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict
  catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not
  semantic. A future ML stage would replace it, but it's good enough for
  a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a
  budget to avoid runaway costs.

## Alternatives considered

- **Do nothing; keep flat FilterConfig.** Rejected ? the umbrella exists
  to dissolve the conflation of quality, topic, and security into a
  single boolean. A pipeline is the natural generalization.
- **Inline each stage in the handler** (no protocol). Rejected ? makes
  it impossible to test stages independently, reorder them, or add new
  ones without touching the hot path.
- **Use a rules engine / DSL.** Rejected ? over-engineered for <100
  users. A Python list of callables is simpler, testable, and
  introspectable.
- **Bundle all phases into a single PR.** Rejected ? scope discipline.
  Each phase is independently shippable; this ADR defines the shared
  interface they code against.

## Open questions for the decision owner

1. **`StageResult.metadata` shape** ? dict or typed per-stage? Recommend:
   dict for Phase 1 (flexible), typed in Phase 2 (once the shape
   stabilizes).
2. **Policy decisions table** ? store every decision, or only failures?
   Recommend: every decision (explainability), with a TTL-based cleanup
   for old traces.
3. **Topic matching** ? substring only, or regex/glob? Recommend:
   substring (case-insensitive) for v1; natural-language matching in a
   follow-up.
4. **Deep inspection budget** ? per-item byte limit or per-account
   bandwidth cap? Recommend: per-item limit (10 MB default) with a
   per-account monthly cap enforced at the app level.

## References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier
  4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept
  generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this
  design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

#### 4. assistant (2026-07-06T19:18:45.119Z)

Now let me verify the handler code and ADR 0009.

#### 5. user (2026-07-06T19:18:47.379Z)

```
         plus the existing (channel, message) instance dedup,
      3. runs the filter engine (read from ``engine_holder`` so config
         changes via ``/set`` take effect immediately),
      4. inserts a VideoRecord row,
      5. delivers a match to ``delivery_target`` (see ADR 0005).
```

```
"""

async def on_new_message(event: NewMessage.Event) -> None:  # pragma: no cover
    message: Message = event.message  # type: ignore[assignment]
    if message is None:
        return  # Telethon edge case: event has no message (shouldn't happen)
    # Only process genuine channel/group posts and private messages we
    # receive ? outgoing messages (sent by our own account) are ignored.
    if getattr(message, "out", False):
        return

    meta = extract_video_meta(message)
    if meta is None:
        return  # not a video

    channel_id = _peer_id(message.peer_id)
    message_id = message.id
    document_id = _extract_document_id(message)

    # --- watched gate (ADR 0009, Track E) -------------------------------
    # Opt-in: a source is ingested only when watched=True. An absent
    # sources row means the channel has never been seen ? treat it as
    # unwatched and lazily create a watched=0 row for /channels
    # discoverability (ADR 0009 lines 37-44, 130-131).
    if not await db.should_ingest_source(conn, user.id, channel_id):
        return  # skip unwatched / unknown source

    # --- three-tier dedup (cheapest first) ---

    # Tier 0: same (user, channel, message) instance ? already processed.
    existing = await db.get_video(conn, user.id, channel_id, message_id)
    if existing is not None:
        return

    # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
    if document_id is not None:
        existing = await db.get_video_by_document_id(conn, user.id, document_id)
        if existing is not None:
            log.info(
                "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
                "original_channel=%s original_msg=%s",
                user.id,
                document_id,
                channel_id,
                message_id,
                existing.channel_id,
                existing.message_id,
            )
            return

    # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
    if (
        meta.size_bytes is not None
        and meta.duration_s is not None
        and meta.height is not None
        and meta.width is not None
    ):
        existing = await db.get_video_by_content_fingerprint(
            conn,
            user.id,
            meta.size_bytes,
            meta.duration_s,
            meta.height,
            meta.width,
        )
```

```
            if existing is not None:
                log.info(
                    "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
                    "channel=%s msg=%s original_channel=%s original_msg=%s",
                    user.id,
                    meta.size_bytes,
                    meta.duration_s,
                    meta.height,
                    meta.width,
                    channel_id,
                    message_id,
```

**6. user (2026-07-06T19:18:48.286Z)**

# ADR 0009: Selective channel scanning and multiplexed routing

- **Status**: Accepted
- **Date**: 2026-07-04
- **Decision owner**: project owner

## Context

Today the userbot scans **every** channel/group the account is a member of. `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=` filter, and `on_new_message` processes every video regardless of source. The `channels` table exists but is populated lazily as a *record* of seen channels; it is not used as a filter.

The product vision is a **multiplexing video re-distribution system**:

- A small set of high-quality **source** channels (e.g. spiritual, classical music) are scanned for incoming videos.
- Matches are routed to themed **destination** broadcast channels (e.g. a Reggae channel, a Rap channel) that subscribers opt into.
- A user therefore subscribes to *outputs* by topic, not to *inputs* by source.

This requires two capabilities the current architecture does not have:

1. **Selective scanning** ? choose which source channels are watched, rather than ingesting all of them.
2. **Per-source ? per-destination routing** ? a match from source channel X is delivered to a specific destination channel Y, instead of the single global `delivery_target` (ADR 0005).

## Decision

Add three additive, independently-shippable capabilities:

### 1. Selective scanning (watched-flag)

Add a `watched` flag to the `channels` table. The default is locked as **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they explicitly `/watch` a channel. This matches the multiplexing product shape (sources are a *curated* set) and avoids blindly re-ingesting every chat the account is a member of.

Gate `on_new_message` at the top: after computing `channel_id`, look up the row and `return` early if `watched` is false, before any dedup or filter work.

This is a small migration plus ~5 lines in `telethon_client.py`.

**Discoverability note.** With `DEFAULT 0` plus the current lazy population (a `channels` row is only inserted when a message is first seen), opt-in is

necessary but not sufficient for discoverability: a channel the account has never received a message from has no row to flag. The `/channels` command (step 2 below) therefore upserts candidate rows from `client.get_dialogs()` so users can browse and watch channels they have not yet received traffic from. The flag default and the enumeration flow are orthogonal ? both are required.

#### 2. Channel enumeration and management commands

New Saved Messages commands (and, via ADR 0008, bot commands) building on the existing `commands.py` pattern:

- `/channels` ? list the user's channels (via `client.get_dialogs()`, already used in `backfill.py`) with their watched state.
- `/watch <#name|id>` ? set `watched=1` for a source channel.
- `/unwatch <#name|id>` ? set `watched=0`.
- `/watchlist` ? list only watched sources.

#### 3. Per-source ? per-destination routing rules

Introduce a `routing_rules` table (the canonical name; issue #25's `routes` refers to the same concept and should be aligned to `routing_rules`), e.g.:

```
routing_rules (
  user_id            INTEGER NOT NULL REFERENCES users(id),
  source_channel_id  INTEGER NOT NULL,
  dest_channel_id    INTEGER NOT NULL,   -- a broadcast channel / @username
  filter_config_id   INTEGER,            -- optional per-route filter override
  PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
)
```

On a match, look up applicable routing rules by `source_channel_id` and deliver to each matching destination. The delivery layer (`delivery.py`) already accepts a `target` per call, so this generalises the existing single `delivery_target` to a per-match lookup. Users with no routing rules fall back to the current global `delivery_target` behaviour (ADR 0005), preserving backwards compatibility.

## Consequences

**Positive**
- Enables the multiplexing product shape: scan curated sources, publish to topical broadcast channels, let subscribers pick outputs by interest.
- Each capability is additive and independently deployable; none disturbs the existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
- Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original* file, so a routed re-upload is not re-processed.
- Channel enumeration reuses `client.get_dialogs()` already present in `backfill.py`; no new Telethon capability is required.

**Negative**
- New schema (`channels.watched`, `routing_rules`) with migrations.
- **Routing loop invariant.** With multiplexed routing, the userbot account may be a member of a *destination* broadcast channel and re-see its own output, producing an infinite forward loop. The contract is: **a channel is either a source or a destination, never both.** This is enforced in `on_new_message` by excluding any channel that is a configured destination (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE user_id = ?)`) before the watched-flag check. Issue #25 Track E implements this; the invariant is stated here so the handler has a contract to code against.
- Per-route filter overrides add config complexity; v1 may ship with global

```
  filters only and a single implicit route (`source ? global target`), then
  layer explicit per-route rules later.
```

## Backwards compatibility

```
Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
install from receiving matches until the user re-watches their channels ? a
```

#### 7. assistant (2026-07-06T19:19:02.935Z)

```
The finding holds strongly. Let me confirm the key facts:

1. **Handler order**: `should_ingest_source` (watched gate) runs at telethon_client.py line
~189, BEFORE all dedup and filter work ? confirmed by the code and the comment "watched gate
(ADR 0009, Track E)".

2. **Pipeline placement**: ADR 0012 explicitly states the pipeline runs "**after** dedup and
**before** delivery" (Context §"What we want" and "Policy evaluation flow (post-dedup, pre-
delivery)"). So the pipeline is strictly downstream of the watched gate.

3. **Tier 3 stage 1**: "Source allowlist: reject items from sources not in the `allowlist` (an
explicit list of trusted provider_source_ids)." ? this is a source-level opt-in gate, exactly
what `watched` does.

Let me consider possible refutations:

- **Could the allowlist be broader than watched?** No. An unwatched source returns early at the
top and never reaches the pipeline. So by the time the allowlist stage runs, the source is
already watched. The allowlist can only ever be a subset of watched sources ? rejects nothing
that isn't already blocked... unless a user watches a source but does NOT allowlist it. In that
case the allowlist WOULD reject items from watched-but-not-allowlisted sources.

Let me examine this potential refutation carefully. The two gates aren't strictly identical in
effect: `watched` is account-level "should I ingest this source at all", while an allowlist
within a policy could reject a source that IS watched. That's a *per-policy* narrowing distinct
from the account-wide ingestion gate ? which is precisely the "orthogonal scope" the finding's
recommendation itself suggests.

So is it truly "dead code / can never fire"? Not strictly. If a user watches source X (so it's
ingested) but the policy's allowlist excludes X, the stage fires and rejects. That's a real,
non-no-op behavior. However, the ADR describes the allowlist as "trusted provider_source_ids"
with no mention of per-policy/per-route scoping ? and given watched is already the source opt-
in, the natural user model is that a watched source is trusted. The design does NOT articulate
what watched-but-not-allowlisted means, nor reconcile the two gates.

Let me verify the ADR's own cross-reference framing.
```

#### 8. user (2026-07-06T19:19:05.693Z)

```
55:?     source allowlist, keyword match, channel topic tags    ?
125:1. **Source allowlist**: reject items from sources not in the `allowlist`
174:- `policies.config_json` gains optional `topics`, `allowlist`,
187:| **F3** ? Source allowlist | `allowlist` config, skip non-allowlisted sources | F1 |
226:  single boolean. A pipeline is the natural generalization.
256:- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept
257:  generalizes `sources.watched`).
```

#### 9. assistant (2026-07-06T19:19:16.307Z)

```
This is decisive. Line 256-257 of the References section states: "ADR 0009 ? selective channel
scanning (the `topics.allowlist` concept **generalizes** `sources.watched`)."

So the ADR author is *aware* of the overlap and explicitly frames the allowlist as a
generalization of `sources.watched`. But this framing is exactly the problem the finding
```

identifies:

- A "generalization" of watched would need to do something watched cannot. But because the pipeline runs strictly downstream of the watched gate (unwatched sources return early and never reach any stage), the allowlist stage can never see an unwatched source. It cannot generalize watched ? it can only re-narrow within the already-watched set.

- The ADR never specifies the semantics of watched-but-not-allowlisted. The F3 scope (line 187) says simply "skip non-allowlisted sources" ? which, if interpreted as the account-level source set, is precisely the watched gate re-implemented one layer down. If interpreted as per-policy narrowing, the ADR doesn't say so and doesn't reconcile it with watched.

Now the adversarial question ? is this a *real* gap or REFUTED because the allowlist could fire on watched-but-not-allowlisted sources? The finding's core claim is nuanced: the specific claim "can NEVER fire / dead code" is too strong (it CAN fire for watched-but-not-allowlisted sources). But the substantive design defect ? undefined interaction between two overlapping opt-in source gates, and a stage the ADR itself calls a "generalization" that structurally cannot generalize because it sits downstream of the gate it duplicates ? genuinely holds. The ADR provides no reconciliation.

Let me also confirm there's no per-policy source scoping already implied that would make the allowlist meaningful and defined.

**10. user (2026-07-06T19:19:18.305Z)**

122:Security stages are **opt-in** and configured per-policy. The design
141:no-download principle. The opt-in is per-policy (`deep_inspect: true` in

**11. assistant (2026-07-06T19:19:30.709Z)**

Confirmed: the ADR never mentions per-route/multi-route source scoping (ADR 0009's `routing_rules`). Security stages are "configured per-policy" (line 122), but there's exactly one policy per account today (is_default=1, one row per account ? per ADR 0012's own Context table). So "per-policy" ? per-account, which is the same granularity as watched. There is no per-route dimension that would give the allowlist independent meaning.

I have enough to render a verdict.

**Assessment:**

The finding's central design point is CONFIRMED: the Tier 3 "Source allowlist" stage overlaps with `sources.watched`, and because the pipeline runs strictly after the watched gate (which returns early for unwatched sources at telethon_client.py:189, before dedup/evaluate/deliver), the allowlist stage cannot generalize watched ? it sits downstream of the very gate it duplicates. The ADR's own References section (lines 256-257) claims the allowlist "generalizes `sources.watched`," which is structurally impossible given the pipeline placement. The two overlapping opt-in source gates are left unreconciled, and F3's scope ("skip non-allowlisted sources") reimplements watched at the same account/policy granularity (there is only one policy per account; no per-route scoping exists in this ADR).

One correction to the finding's phrasing: the strong "dead code / can never fire" claim is slightly overstated. The stage *can* fire for a source that is watched but excluded from the policy allowlist. But that behavior is undefined by the ADR and, at the current one-policy-per-account granularity, is a confusing re-narrowing that duplicates the source-gate concept rather than adding an orthogonal capability. The recommendation (drop the stage or rescope it to per-route restriction using ADR 0009's `routing_rules`) is sound and directly addresses the real gap.

Severity: the finding says high. Calibrating for a draft design doc: this is not a false factual claim that breaks an F-phase or causes data loss ? F3 would still "work," just redundantly/confusingly. It's an unreconciled overlap and an inaccurate self-cross-reference (the "generalizes" claim). That is a genuine design gap/inconsistency the owner should resolve ? **medium**.

**12. user (2026-07-06T19:19:42.732Z)**

```
Structured output provided successfully
```